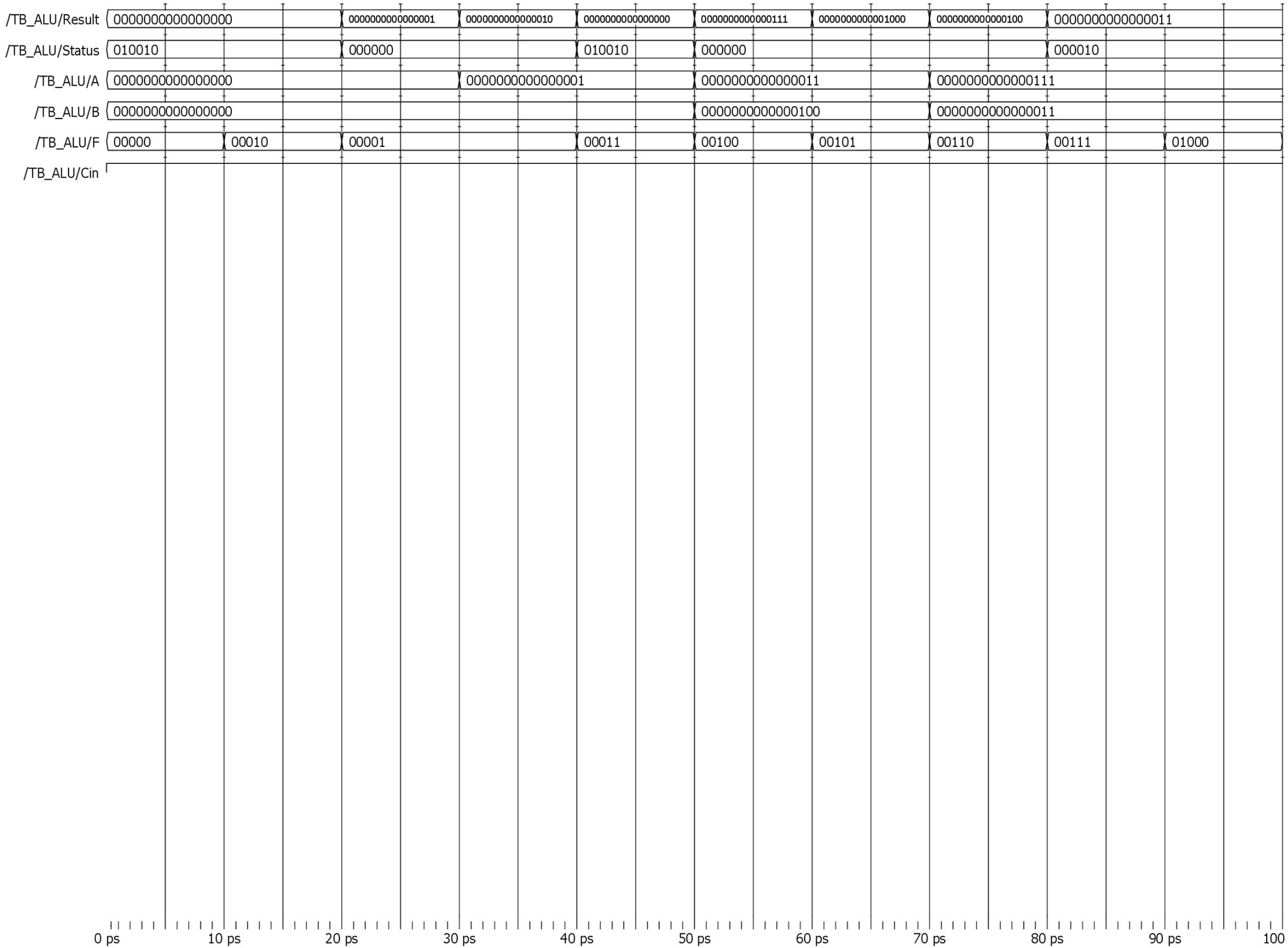
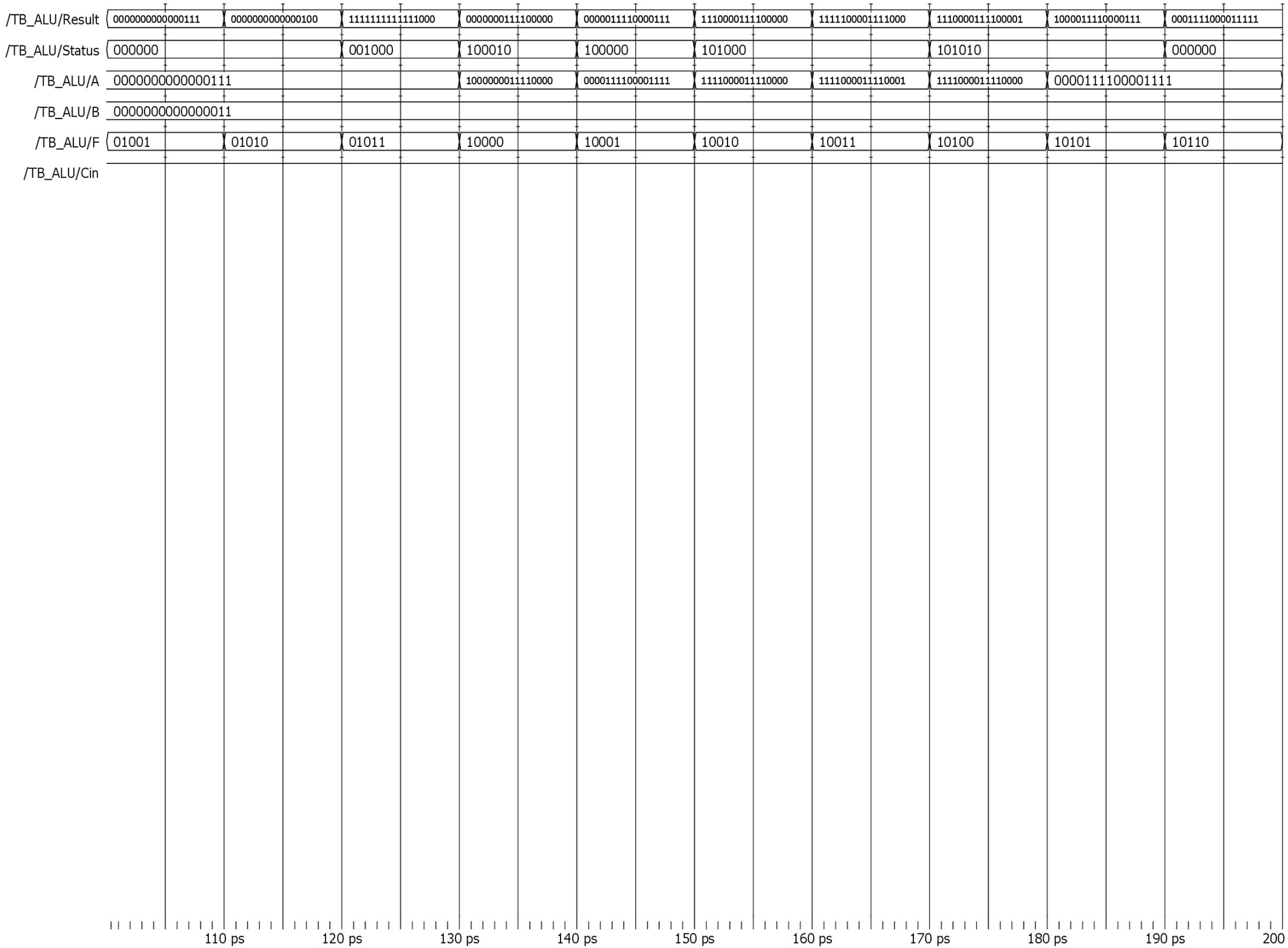
```
module ALU(output reg [15:0]Result,output reg [5:0]Status,
            input [15:0] A,B,
            input [4:0] F,
            input Cin);
    // All functions Needed in ALU
    localparam   [4:0]
                INC = 5'b00001,
                DEC = 5'b00011,
                ADD = 5'b00100,
                ADD_CARRY = 5'b00101,
                SUB = 5'b00110,
                SUB_BORROW = 5'b00111,
                AND = 5'b01000,
                OR = 5'b01001,
                XOR = 5'b01010,
                NOT = 5'b01011,
                SHL = 5'b10000,
                SHR = 5'b10001,
                SAL = 5'b10010,
                SAR = 5'b10011,
                ROL = 5'b10100,
                ROR = 5'b10101,
                RCL = 5'b10110,
                RCR = 5'b10111;
    // FLags
    localparam
                CARRY_F = 5,
                ZERO_F = 4,
                NEGATIVE_F = 3,
                OVERFLOW_F = 2,
                PARITY_F = 1,
                AUX_CARRY_F = 0;
    // Internal signals for AUX carry (SUB operations)
    wire [4:0] sub_aux = {1'b0, A[3:0]} - {1'b0, B[3:0]};
    wire [4:0] sub_borrow_aux = {1'b0, A[3:0]} - {1'b0, B[3:0]} - {4'b0, Cin};
    wire [4:0] dec_aux = {1'b0, A[3:0]} - 5'h1;
    // Internal signals for AUX carry (ADD operations)
    wire [4:0] add_aux = {1'b0, A[3:0]} + {1'b0, B[3:0]};
    wire [4:0] add_carry_aux = {1'b0, A[3:0]} + {1'b0, B[3:0]} + {4'b0, Cin};
    wire [4:0] inc_aux = {1'b0, A[3:0]} + 5'h1;
    // Internal signals for arithmetic operations
    wire [16:0] add_result = A + B + ((F == ADD_CARRY) ? Cin : 1'b0);
    wire [16:0] sub_result = A - B - ((F == SUB_BORROW) ? Cin : 1'b0);
    wire [16:0] inc_result = A + 17'h00001;
    wire [16:0] dec_result = A - 17'h00001;
    // Result Setting
    always @(*) begin
        case (F)
            INC : Result = A + 16'h0001;
            DEC : Result = A - 16'h0001;
            ADD : Result = A + B;
            ADD_CARRY : Result = A + B + Cin;
            SUB : Result = A - B;
            SUB_BORROW : Result = A - B - Cin;
            AND : Result = A & B;
            OR : Result =  A | B;
            XOR : Result = A ^ B;
            NOT : Result = ~A;
            SHL : Result = A << 1;
```

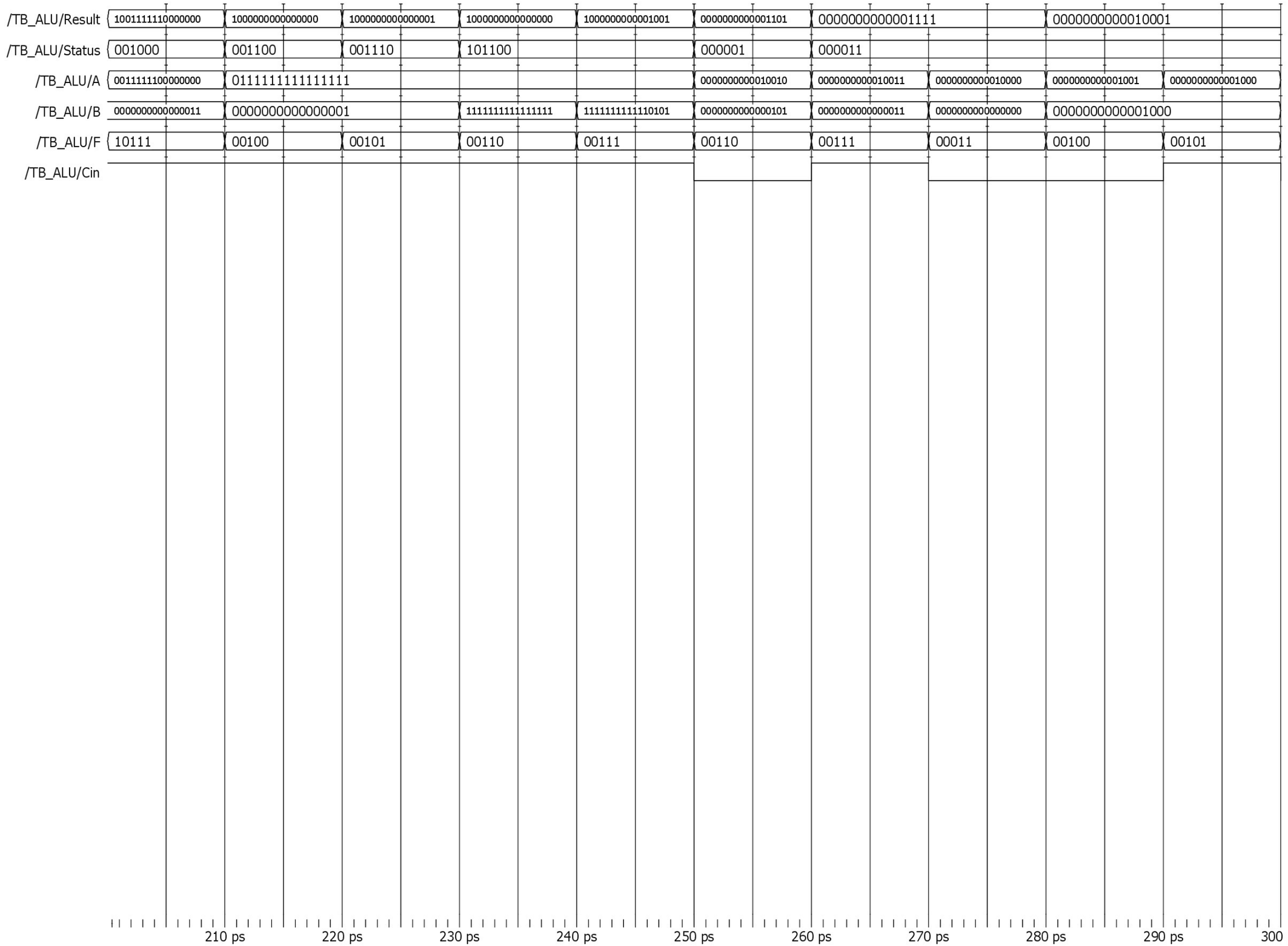
```
            SHR : Result = A >> 1;
            SAL : Result = A <<< 1;
            SAR : Result = {A[15], A[15:1]};
            ROL : Result = {A[14:0],A[15]};
            ROR : Result = {A[0],A[15:1]};
            RCL : Result = {A[14:0],Cin};
            RCR : Result = {Cin,A[15:1]};
            default: Result = 16'h0000;
        endcase
    end
    // Status Setting
    always @(*) begin
        Status[ZERO_F] = (Result == 16'h0000);
        Status[NEGATIVE_F] = Result[15];              // MSB
        Status[PARITY_F] = ~(^Result);
        Status[CARRY_F] = 1'b0;
        Status[OVERFLOW_F] = 1'b0;
        Status[AUX_CARRY_F] = 1'b0;
        case (F)
            ADD,ADD_CARRY,SUB,SUB_BORROW,INC,DEC: begin
                // SUB Operation
                if(F[1]) begin
                    Status[CARRY_F] = (F == DEC) ? dec_result[16] : sub_result[16];
                    Status[OVERFLOW_F] = (A[15] != B[15]) && (Result[15] != A[15]);
                    case (F)
                        SUB:      Status[AUX_CARRY_F] = sub_aux[4];  // Borrow from nibble
                        SUB_BORROW: Status[AUX_CARRY_F] = sub_borrow_aux[4];
                        DEC:      Status[AUX_CARRY_F] = dec_aux[4];
                        default:  Status[AUX_CARRY_F] = 1'b0;
                    endcase
                end
                // ADD Operation
                else begin
                    Status[CARRY_F] = (F == INC) ? inc_result[16] : add_result[16];
                    Status[OVERFLOW_F] = (A[15] == B[15]) && (Result[15] != A[15]);
                    case (F)
                        ADD:      Status[AUX_CARRY_F] = add_aux[4];  // Carry from nibble
                        ADD_CARRY: Status[AUX_CARRY_F] = add_carry_aux[4];
                        INC:      Status[AUX_CARRY_F] = inc_aux[4];
                        default:  Status[AUX_CARRY_F] = 1'b0;
                    endcase
                end
            end
            ROL,RCL,SHL,SAL:begin
                Status[CARRY_F] = A[15];
            end
            ROR,RCR,SAR,SHR:begin
                Status[CARRY_F] = A[0];
            end
            default: begin
                Status[CARRY_F] = 1'b0;
                Status[OVERFLOW_F] = 1'b0;
                Status[AUX_CARRY_F] = 1'b0;
            end
        endcase
    end
endmodule

`timescale 1ps/1ps
module TB_ALU;
```

```
wire [15:0] Result;
wire [5:0] Status;
reg [15:0] A,B;
reg [4:0] F;
reg Cin;
ALU ALU_BlockTest (.Result(Result),.Status(Status),.A(A),.B(B),.F(F),.Cin(Cin));
initial begin
    #320 $finish;
end
initial begin
    $dumpfile("ALU_TESTBENCH.vcd");
    $dumpvars(0,TB_ALU);
end
initial fork
    // operations
    #0 {A,B,F,Cin} = {16'h0000,16'h0000,5'b00000,1'b1};
    #10 {A,B,F,Cin} = {16'h0000,16'h0000,5'b00010,1'b1};
    #20 {A,B,F,Cin} = {16'h0000,16'h0000,5'b00001,1'b1};
    #30 {A,B,F,Cin} = {16'h0001,16'h0000,5'b00001,1'b1};
    #40 {A,B,F,Cin} = {16'h0001,16'h0000,5'b00011,1'b1};
    #50 {A,B,F,Cin} = {16'h0003,16'h0004,5'b00100,1'b1};
    #60 {A,B,F,Cin} = {16'h0003,16'h0004,5'b00101,1'b1};
    #70 {A,B,F,Cin} = {16'h0007,16'h0003,5'b00110,1'b1};
    #80 {A,B,F,Cin} = {16'h0007,16'h0003,5'b00111,1'b1};
    #90 {A,B,F,Cin} = {16'h0007,16'h0003,5'b01000,1'b1};
    #100 {A,B,F,Cin} = {16'h0007,16'h0003,5'b01001,1'b1};
    #110 {A,B,F,Cin} = {16'h0007,16'h0003,5'b01010,1'b1};
    #120 {A,B,F,Cin} = {16'h0007,16'h0003,5'b01011,1'b1};
    #130 {A,B,F,Cin} = {16'h80f0,16'h0003,5'b10000,1'b1};
    #140 {A,B,F,Cin} = {16'h0f0f,16'h0003,5'b10001,1'b1};
    #150 {A,B,F,Cin} = {16'hf0f0,16'h0003,5'b10010,1'b1};
    #160 {A,B,F,Cin} = {16'hf0f1,16'h0003,5'b10011,1'b1};
    #170 {A,B,F,Cin} = {16'hf0f0,16'h0003,5'b10100,1'b1};
    #180 {A,B,F,Cin} = {16'h0f0f,16'h0003,5'b10101,1'b1};
    #190 {A,B,F,Cin} = {16'h0f0f,16'h0003,5'b10110,1'b1};
    #200 {A,B,F,Cin} = {16'h3f00,16'h0003,5'b10111,1'b1};
    // Overflow FLag Testing
    #210 {A,B,F,Cin} = {16'h7fff,16'h0001,5'b00100,1'b1};
    #220 {A,B,F,Cin} = {16'h7fff,16'h0001,5'b00101,1'b1};
    #230 {A,B,F,Cin} = {16'h7fff,16'hffff,5'b00110,1'b1};
    #240 {A,B,F,Cin} = {16'h7fff,16'hfff5,5'b00111,1'b1};
    // aux.  flag testing
    #250 {A,B,F,Cin} = {16'h0012,16'h0005,5'b00110,1'b0};
    #260 {A,B,F,Cin} = {16'h0013,16'h0003,5'b00111,1'b1};
    #270 {A,B,F,Cin} = {16'h0010,16'h0000,5'b00011,1'b0};
    #280 {A,B,F,Cin} = {16'h0009,16'h0008,5'b00100,1'b0};
    #290 {A,B,F,Cin} = {16'h0008,16'h0008,5'b00101,1'b1};
    #300 {A,B,F,Cin} = {16'h000F,16'h0000,5'b00001,1'b0};
join
endmodule
```

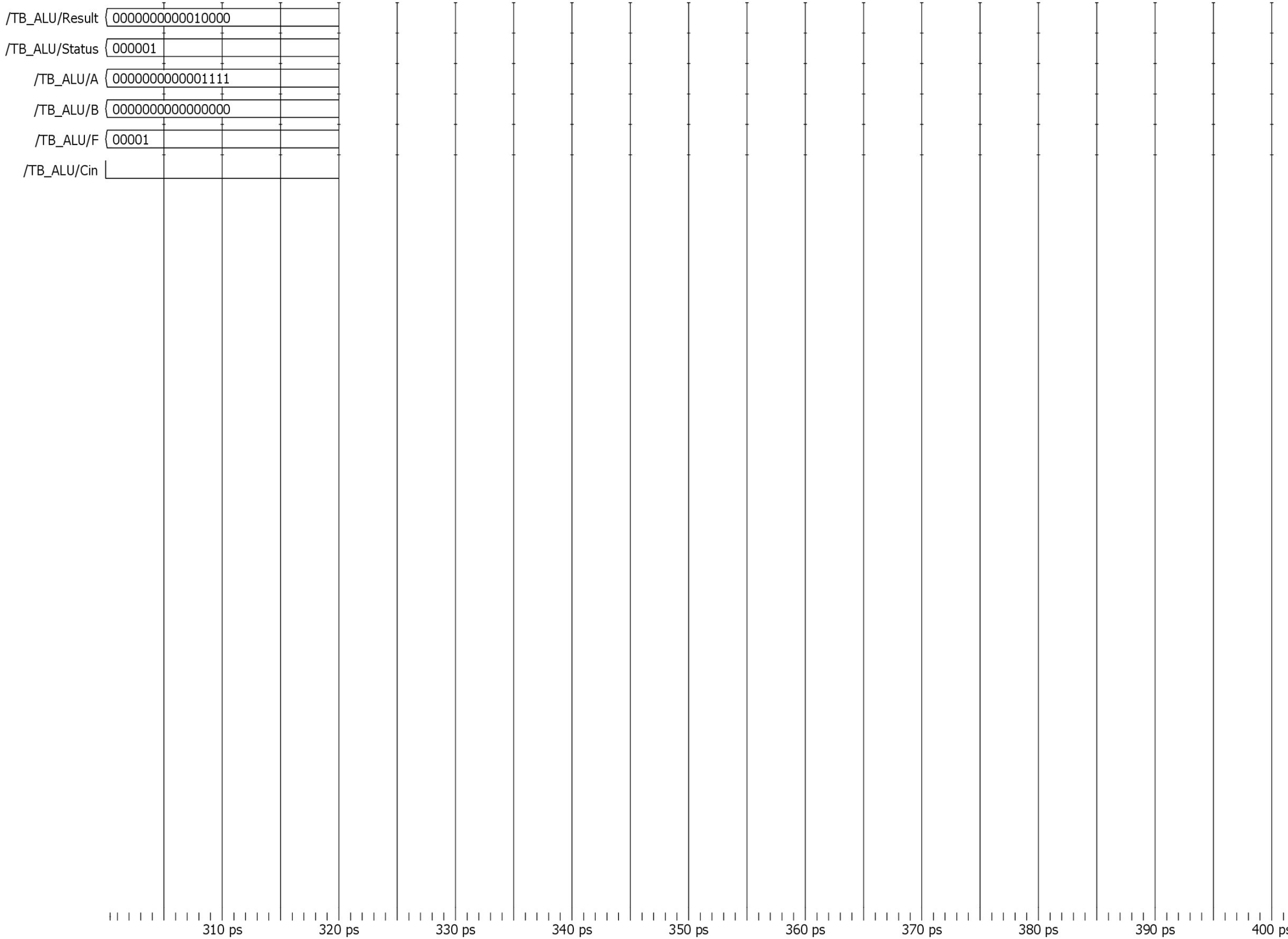


Entity:TB_ALU Architecture: Date: Tue Nov 25 18:32:40 +0200 2025 Row: 1 Page: 1


/TB_ALU/Result
0000000000000111
0000000000000100
1111111111111000
0000000111100000
0000011110000111
1110000111100000
1111100001111000
1110000111100001
1000011110000111
0001111000011111
/TB_ALU/Status
000000
001000
100010
100000
101000
101010
000000
/TB_ALU/A
0000000000000111
1000000011110000
0000111100001111
1111000011110000
1111000011110001
1111000011110000
0000111100001111
/TB_ALU/B
0000000000000011
/TB_ALU/F
01001
01010
01011
10000
10001
10010
10011
10100
10101
10110
/TB_ALU/Cin
110 ps
120 ps
130 ps
140 ps
150 ps
160 ps
170 ps
180 ps
190 ps
200 ps



Entity:TB_ALU Architecture: Date: Tue Nov 25 18:34:45 +0200 2025 Row: 1 Page: 1



Entity:TB_ALU Architecture: Date: Tue Nov 25 18:35:25 +0200 2025 Row: 1 Page: 1